

Query Analysis Report

Project: ULK Academic Activity and Resource Management System

Phase 7: Query Processing Analysis

1. 1. Introduction This report analyzes three complex queries executed on the PostgreSQL database system. The analysis focuses on execution plans, join strategies, and computational overhead.

2. 2. Query 1: Join of Multiple Tables SQL: 'sql SELECT s.full_name, s.student_code, e.grade FROM Student s JOIN Enrollment e ON s.id = e.student_id JOIN Course c ON e.course_id = c.id WHERE c.title = 'Advanced Databases Part 3'; '

2.1. Execution Plan Analysis 1. **Filter Execution:** PostgreSQL first performs a Sequential Scan (or Index Scan if an index exists) on the Course table to find the record where title = 'Advanced Databases Part 3'. 2. **First Join (Hash Join):** The result from the Course filter is used as a hash table. The Enrollment table is scanned, and matches on course_id are identified. 3. **Second Join (Nested Loop):** The filtered enrollments are then joined with the Student table. Since Student.id is a primary key, a Nested Loop Join with an Index Scan on Student_pkey is performed. 4. **Projection:** Final columns full_name, student_code, and grade are returned.

Interpretation: The efficiency of this query depends heavily on the filter on Course.title. Without an index, a full table scan of Course is required. The use of PK/FK relationships allows PostgreSQL to optimize the joins using indexed lookups.

3. 3. Query 2: Nested Aggregation SQL: 'sql SELECT d.name, AVG(e.grade) as dept_avg FROM Department d JOIN Student s ON d.id = s.department_id JOIN Enrollment e ON s.id = e.student_id GROUP BY d.name ORDER BY dept_avg DESC LIMIT 1; '

3.1. Execution Plan Analysis 1. **Join Sequence:** The query joins Department, Student, and Enrollment. 2. **Grouping (HashAggregate):** PostgreSQL uses a HashAggregate to group records by department.name. It maintains a running sum and count for each group to calculate the average. 3. **Sorting (Sort):** After aggregation, the results are sorted in descending order of the average grade. 4. **Limit:** The top 1 record is retrieved after the sort.

Interpretation: This is a resource-intensive query because it involves a full scan of the Enrollment table. The HashAggregate is efficient for a small number of departments, but if there were thousands, it might spill to disk (external sort).

4. 4. Query 3: Window Function SQL: 'sql SELECT c.title, s.full_name, e.grade, RANK() OVER (PARTITION BY c.id ORDER BY e.grade DESC) as rank_in_course FROM Course c JOIN Enrollment e ON c.id = e.course_id JOIN Student s ON e.student_id = s.id; '

4.1. Execution Plan Analysis 1. **Joins:** Performs joins between Course, Enrollment, and Student. 2. **WindowAgg Node:** PostgreSQL introduces a WindowAgg node in the plan. 3. **Internal Sort:** To process the PARTITION BY c.id ORDER BY e.grade DESC clause, the entire joined dataset must be sorted first by course_id and then by grade within each course. 4. **Ranking:** The rank is calculated by iterating

through the sorted results.

Interpretation: Window functions are powerful but require sorting the entire result set. The cost is primarily in the Sort operation. Indexing Enrollment(course_id, grade DESC) could significantly improve performance by allowing the WindowAgg to skip the explicit sort.

5. 5. Conclusion PostgreSQL's query optimizer effectively chooses between Hash Joins and Nested Loops based on table sizes and available indexes. Complex analytical queries involving window functions and aggregations highlight the importance of proper indexing on joining and sorting columns.